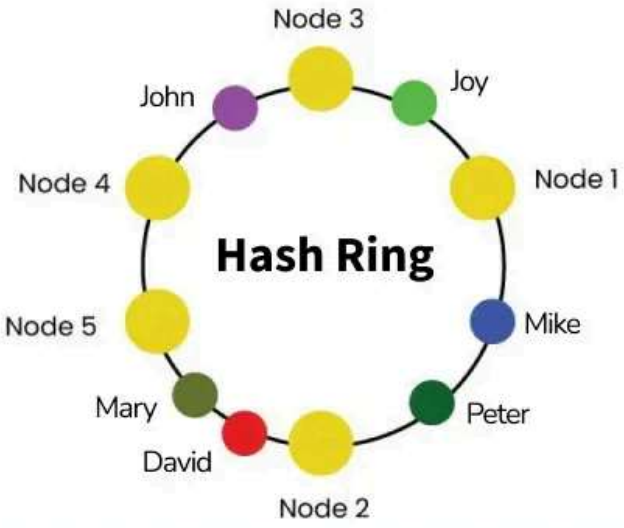
Consistent Hashing - System Design

Last Updated : 07 Aug, 2025

Consistent hashing is a distributed hashing technique used in load balancing. The goal is to minimize the need for rehashing when the number of nodes in a system changes.

- It represents the requests by the clients and the server nodes in a virtual ring structure which is known as a **hashring**.
- The number of locations in this ring is not fixed, but it is considered to have an infinite number of points
- The server nodes can be placed at random locations on this ring which can be done using hashing.
- The requests are also placed on the same ring using the same hash function.

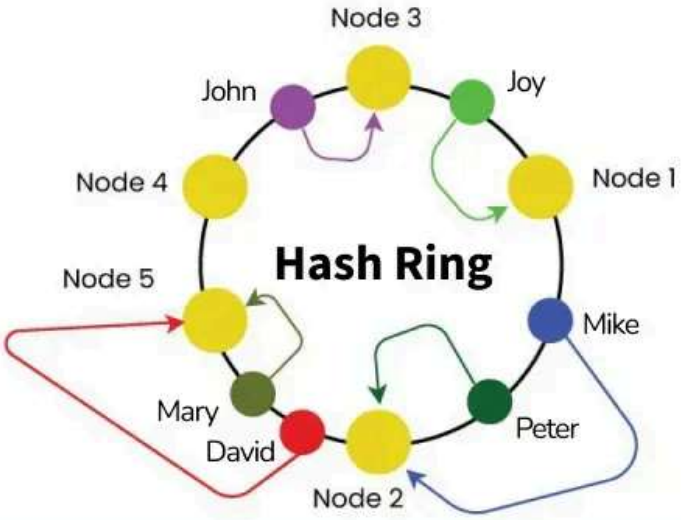


Consistent Hashing

How to decide which request will be served by which server?

If we assume the ring is ordered so that the clockwise traversal of the ring corresponds to the increasing order of location addresses,

so each request can be served by the server node that first appears while traversing clockwise.



Mapping in the hashing

Issues with Traditional Hashing Methods

Here are some key issues with traditional hashing methods explained in easy language:

- **Uneven Distribution of Data**: Traditional hashing methods often lead to an uneven distribution of data across servers. When you use a simple hash function, some servers may get more data, while others get very little.
- **Scalability Problems**: A sizable amount of the data must be redistributed among all servers in classic hashing whenever a server (node) is added or removed. As a result, practically all data must be rehashed and reassigned, which is ineffective and results in delays or downtime.
- **Inflexibility with Changing Number of Servers**: If your application requires scaling up by adding more servers or scaling down by removing some, traditional hashing methods struggle to adapt. The entire system becomes unstable, and large amounts of data need to be moved.
- **Node Failure Handling**: When a server fails in a traditional hashing setup, all the data on that server becomes inaccessible until the server

is back up or the data is redistributed. There is no good way to handle node failures.

- **Overhead of Rehashing**: When the system grows or shrinks, traditional hashing requires rehashing of most keys to different servers, which causes a high amount of overhead.

What is the use of Consistent Hashing?

Consistent hashing is a popular technique used in distributed systems to address the challenge of efficiently distributing keys or data elements across multiple nodes/servers in a network. Consistent hashing's primary objective is to reduce the number of remapping operations necessary when adding or removing nodes from the network, which contributes to the stability and dependability of the system.

- Consistent hashing can be used in to share the burden among nodes and lessen the effects of node failures.
- For example, when a new node is added to the network, only a small number of keys are remapped to the new node, which helps to reduce the overhead associated with the addition.
- Similarly, when a node fails, only a small number of keys are affected, which helps to minimize the impact of the failure on the system as a whole.
- Consistent hashing is also useful in ensuring data availability and consistency in a distributed system.

Phases/Working of Consistent Hashing

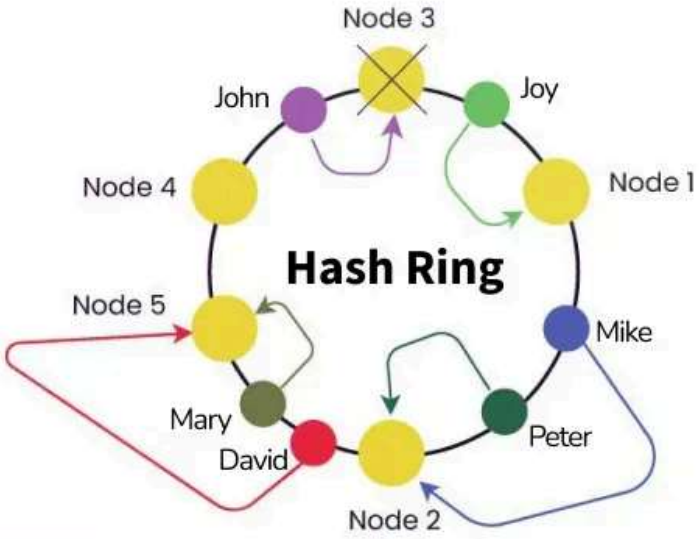
The following are the phases involved in the process of consistent hashing:

- **Phase 1: Hash Function Selection:** Selecting the hash algorithm to link keys to network nodes is the first stage in consistent hashing. This hash function should be deterministic and produce a different value for each key. The selected hash function will be used to map keys to nodes in a consistent and predictable manner.

- **Phase 2: Node Assignment:** Based on the hash function's findings, nodes in the network are given keys in this phase. The nodes are organized in a circle, and the keys are given to the node that is situated closest to the key's hash value in a clockwise direction in the circle.
- **Phase 3: Key Replication:** It's critical to make sure that data is accessible in a distributed system even in the case of node failures. Keys can be copied across a number of network nodes to accomplish this. In the event that one node fails, this helps to guarantee that data is always accessible.
- **Phase 4: Node Addition/Removal:** It can be required to remap the keys to new nodes in order to maintain system balance when nodes are added to or deleted from the network. By only remapping just a small number of keys to the new node, consistent hashing minimizes the impact of added or deleted nodes.
- **Phase 5: Load balancing:** Consistent hashing helps in distributing the load among the network's nodes. To keep the system balanced and effective when a node is overloaded, portions of its keys can be remapped to other nodes.
- **Phase 6: Failure Recovery:** If a node fails, the keys that are assigned to it can be remapped to other nodes in the network. This enables data to remain accurate and always available, even in the case of a node failure.

For example:

Let's say we have 5 nodes in the ring and say node 3 fails, then the range of the next server node widens and any request coming in all of this range, goes to the new server node. This shows that due to use of consistent hashing only a small portion of keys are affected.



Node Failure Example

Implementation of Consistent Hashing algorithm

- **Step 1: Choose a Hash Function:**
  - Select a hash function that produces a uniformly distributed range of hash values. Common choices include MD5, SHA-1, or SHA-256.
- **Step 2: Define the Hash Ring:**
  - Represent the range of hash values as a ring. This ring should cover the entire possible range of hash values and be evenly distributed.
- **Step 3: Assign Nodes to the Ring:**
  - Assign each node in the system a position on the hash ring. This is typically done by hashing the node's identifier using the chosen hash function.
- **Step 4: Key Mapping:**
  - When a key needs to be stored or retrieved, hash the key using the chosen hash function to obtain a hash value.
  - Find the position on the hash ring where the hash value falls.
  - Walk clockwise on the ring to find the first node encountered. This node becomes the owner of the key.
- **Step 5: Node Additions:**

- When a new node is added, compute its position on the hash ring using the hash function.
- Identify the range of keys that will be owned by the new node. This typically involves finding the predecessor node on the ring.
- Update the ring to include the new node and remap the affected keys to the new node.

- **Step 6: Node Removals:**
  - When a node is removed, identify its position on the hash ring.
  - Identify the range of keys that will be affected by the removal. This typically involves finding the successor node on the ring.
  - Update the ring to exclude the removed node and remap the affected keys to the successor node.
- **Step 7: Load Balancing:**
  - Periodically check the load on each node by monitoring the number of keys it owns.
  - If there is an imbalance, consider redistributing some keys to achieve a more even distribution.

Below is an example implementation of Consistent Hashing:

```
#include <bits/stdc++.h>

using namespace std;

class ConsistentHashRing {
private:
    map<int, string> ring;
    set<int> sorted_keys;
    int replicas;

    int get_hash(const string& value) {
        hash<string> hash_function;
        return hash_function(value);
    }
```

```
public:
    ConsistentHashRing(int replicas = 3) : replicas(replicas) {}

      // Function to add Node in the ring
    void add_node(const string& node) {
        for (int i = 0; i < replicas; ++i) {
            int replica_key = get_hash(node + "_" + to_string(i));
            ring[replica_key] = node;
            sorted_keys.insert(replica_key);
        }
    }

      // Function to remove Node from the ring
    void remove_node(const string& node) {
        for (int i = 0; i < replicas; ++i) {
            int replica_key = get_hash(node + "_" + to_string(i));
            ring.erase(replica_key);
            sorted_keys.erase(replica_key);
        }
    }

    string get_node(const string& key) {
        if (ring.empty()) {
            return "";
        }

        int hash_value = get_hash(key);
        auto it = sorted_keys.lower_bound(hash_value);

        if (it == sorted_keys.end()) {
            // Wrap around to the beginning of the ring
            it = sorted_keys.begin();
        }

        return ring[*it];
    }
};

int main() {
    ConsistentHashRing hash_ring;
```

```
    // Add nodes to the ring
    hash_ring.add_node("Node_A");
    hash_ring.add_node("Node_B");
    hash_ring.add_node("Node_C");

    // Get the node for a key
    string key = "first_key";
    string node = hash_ring.get_node(key);

    cout << "The key '" << key << "' is mapped to node: " << node <
endl;

    return 0;
}
```

Output

```
The key 'first_key' is mapped to node: Node_C
```

Note: This example uses a simple hash function and a binary search to find the position on the ring.

Advantages of using Consistent Hashing

Below are some of the key advantages of using consistent hashing:

1. **Load balancing:** Even as the volume of data grows and evolves over time, consistent hashing maintains the system's efficiency and responsiveness by distributing the network's workload among its nodes in a balanced way.
2. **Scalability:** Because consistent hashing is so scalable, it can adjust to variations in the number of nodes or volume of data being processed with negligible to no impact on the system's overall performance.
3. **Minimal Remapping:** By minimizing the amount of keys that need to be remapped whenever a node is added or withdrawn, consistent hashing makes sure that the system remains stable and reliable even as the network evolves.

4. **Increased Failure Tolerance:** Consistent hashing makes data always accessible and current, even in the case of node failures. The stability and dependability of the system as a whole are enhanced by the capacity to replicate keys across several nodes and remap them to different nodes in the event of failure.
5. **Simplified Operations:** The act of adding or removing nodes from the network is made easier by consistent hashing, which makes it simpler to administer and maintain a sizable distributed system.

Disadvantages of using Consistent Hashing

1. **Hash Function Complexity:** The effectiveness of consistent hashing depends on the use of a suitable hash function. The hash function must produce a unique value for each key and be deterministic in order to be useful. The system's overall effectiveness and efficiency may be affected by how complicated the hash function is.
2. **Performance Cost:** The computing resources needed to map keys to nodes, replicate keys, and remap keys in the event of node additions or removals can result in some performance overhead when using consistent hashing.
3. **Lack of Flexibility:** In some circumstances, the system's ability to adapt to changing requirements or shifting network conditions may be constrained by the rigid limits of consistent hashing.
4. **High Resource Use:** As nodes are added to or deleted from the network, consistent hashing may occasionally result in high resource utilization. This can have an effect on the system's overall performance and efficacy.
5. **The complexity of Management:** Managing and maintaining a system that uses consistent hashing can be difficult and demanding, and it often calls for particular expertise and abilities.

Consistent Hashing – System Design

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Hyderabad
Kolkata

GfG 160
System Design